

Understanding Decision Trees: Splitting Criteria, Overfitting, and Pruning

Machine Learning Assignment

Student Name: Nikhitha Tileti

Student ID: 24128365

GitHub Repo: <https://github.com/tiletinikhitha-ops/Machine-Learning-Assignment.git>

Decision Trees in 3 Easy Steps – Splitting Criteria, Overfitting & Pruning

Overview

This is a very useful machine learning technique that can also be used in two cases: classification and regression. This tutorial delves into the basic principles of decision trees, including how they segment data based on criteria like Gini index and entropy. The tutorial illustrates how the decision trees are trained, validated, and visualized via practical implementation on the Iris dataset.

Furthermore, pivotal challenges like overfitting are discussed along with methods such as cost-complexity pruning that help us enhance model generalization. This tutorial has tried to provide a clear approach towards understanding decision trees with code examples and visual analysis.

1. Introduction

Decision Tree is a supervised machine learning algorithm that can be used for solving both classification as well as regression problems. They are essentially tree structures where each internal node is a condition on a feature, each branch is the output of that condition, and each leaf node is a final prediction.

Decision trees have the advantage of being interpretable. Decision trees also have the advantage of being interpretable compared to many other complex machine learning models; they can be visualized and therefore used in applications where transparency is desirable. Decision trees are widely used in areas such as medical diagnosis, financial decision-making, and risk analysis.

The basic concept of decision trees is to divide the dataset recursively into shorter subsets based on feature values that gives more homogeneous groups in terms of labels. This process repeats until a stopping condition occurs, like a maximum depth or minimum amount of samples in a node.

This tutorial covers how decision trees make these splits, the areas of performance that may be affected in a model based on what splitting criteria is used, and pruning techniques to tackle problems such as overfitting.

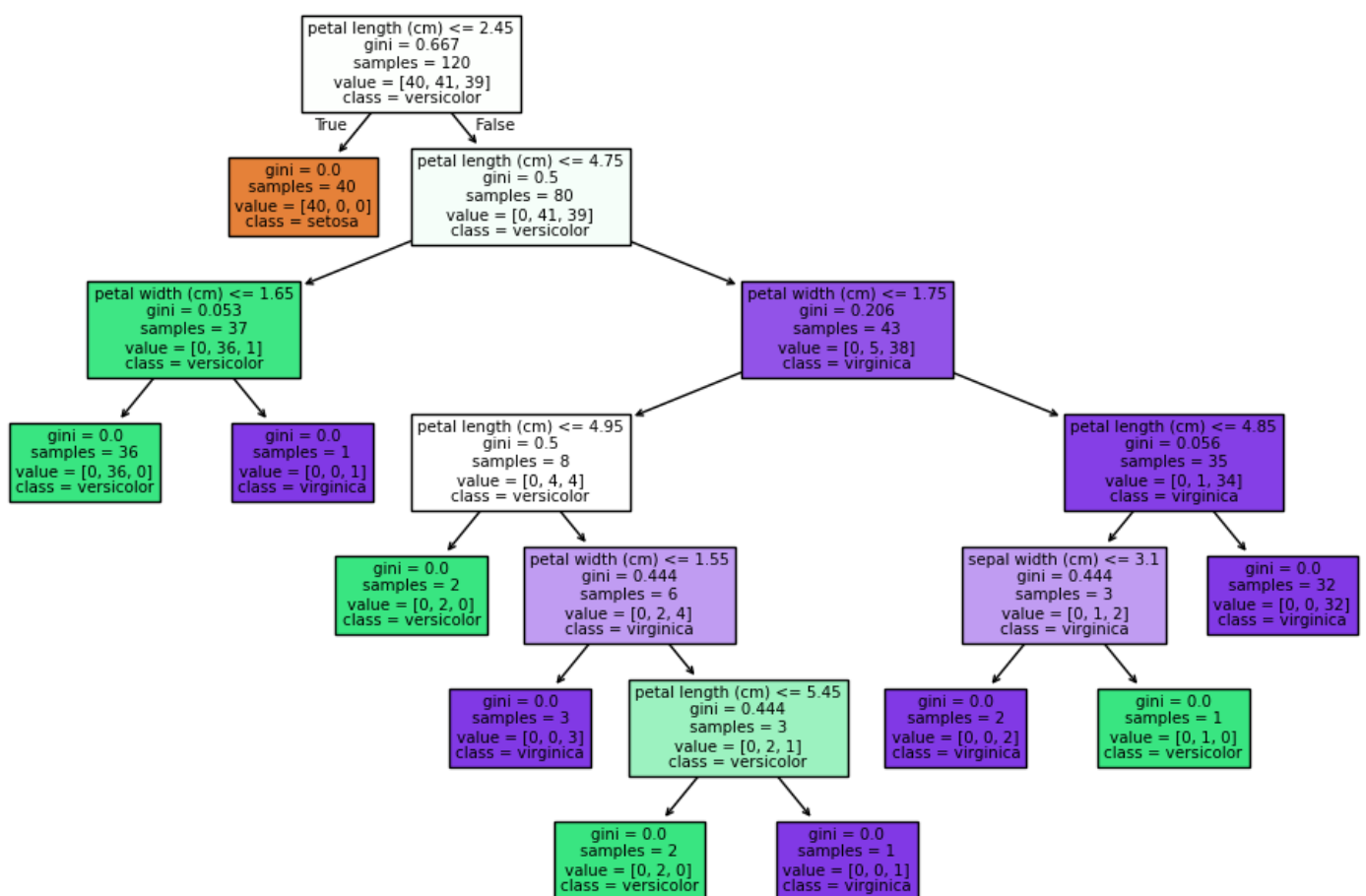
2. How Decision Trees Work

Decision trees are supervised learning models that predict an output by splitting the dataset in a recursive manner into smaller and smaller sub-datasets based on feature values. Decision trees have a flowchart-like structure where each node denotes a decision and branches represent the outcomes of that decision.

This process starts from the root node i.e. a node which holds the complete dataset. How does it work? The algorithm chooses a feature and a threshold that best separates the data into two or more subsets. The algorithm selects the split that maximises the homogeneity of the resulting groups; in other words, as similar as possible data points are grouped by class labels.

These decision rules would be represented by internal nodes, e.g. is the feature value less than or equal to some threshold? Direction of the data to different branches of tree, by the hatchet based on result. This process is repeated recursively until a stopping criterion is reached, such as a maximum depth of the tree or a minimum number of samples at the node.

Figure 1: Visual representation of the trained decision tree.



The leaf nodes signify the model's final predictions. Functionality each leaf assigns class label based on the most frequent occurrence of a class (majority class) in the samples inside that node.

The decision tree diagram shows the connections between various nodes and splits, where the model makes a series of simple decisions based on popular features such as petal length and petal width (Lin et al., 2018).

The decisions progressively split the training set into smaller, more homogeneous groups of samples until they fall into classes.

The hierarchical structure makes decision trees capable of modelling very complex boundaries while keeping the advantages of interpretability and visualisation.

3. Gini Index & Entropy: Criteria to Split

A decision tree is a mechanism to categorize data points into differing groups, one of the vital elements of decision trees is how data should be partitioned at each node. The aim of each split is to generate data subsets that are as pure as possible; samples in each subset should mostly come from the same class. In order to do this, decision trees track impurity measures that are most often the Gini index and entropy.

The gini index: the probability of misclassifying a randomly chosen sample if we randomly labelled it according to the distribution of classes in the node. Hence, a small Gini value means the node is pure so that most of samples belong to one class. A Gini of 0 means completely pure node.

Entropy, in contrast, quantifies the degree of uncertainty or disorder that a given dataset exhibits. It draws on ideas from information theory. Entropy has a higher value when more randomness is present and entropy equals to 0 means all the sample belongs to same class. Target: Improve the overall information gain, which is a measure of how much will the uncertainty be reduced after a split.

Gini index and entropy, both are used to guide the tree in making the most informative splits but they may create tiny different structures for trees. While the distinct criteria often yield similar results, their theoretical explanations allow us to understand the inner workings of a classifier.

In this tutorial, we experimentally implement and compare both criteria. The findings show how each approach has an impact on the shape of the tree and its accuracy on the set.

4. Experimental Setup

Here we showed the behaviour and performance of decision trees using one of the most widely used datasets for classification tasks i.e. Iris data-set during our experiments. These samples include features that are numeric, consisting of 4 numerical features about iris flowers (sepal length, sepal width, petal length and petal width). You are required to identify which species each sample belongs to, among three options: setosa, versicolor, or virginica.

The dataset which was split into training and testing sets using a 80:20 split. Then, the decision tree model was built on the training set and its performance validated through a testing set that it had not previously seen. This helps verify that this model could generalise properly.

Python with scikit-learn library was used for all experiments. Matplotlib was used to produce some visualisations of the model behaviour under different degrees/levels of dither.

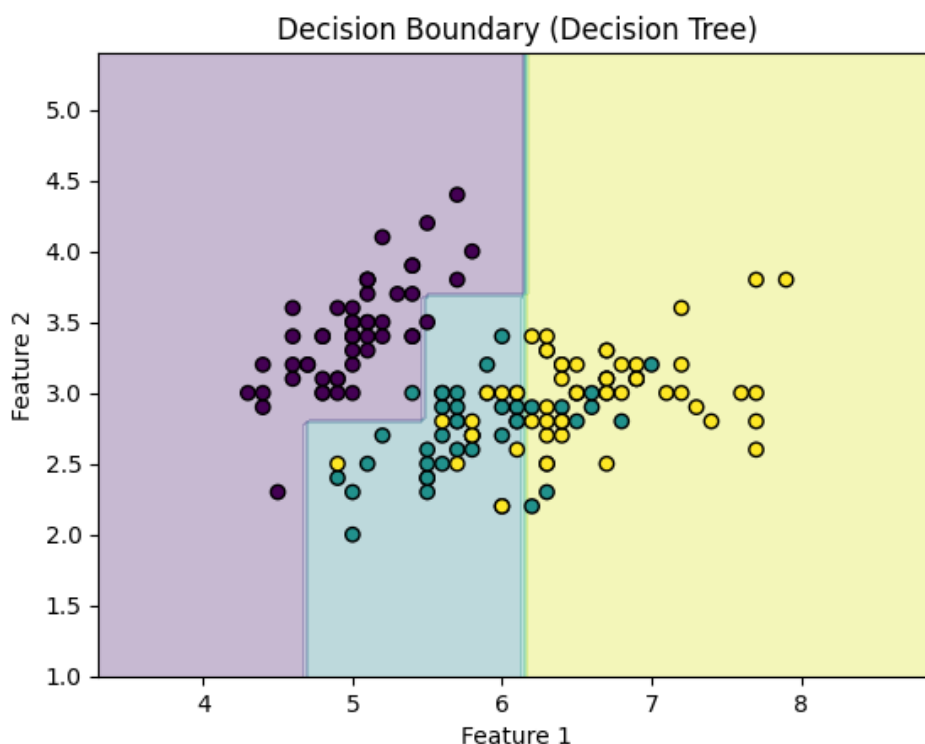
Using the Gini index and entropy as splitting criteria, a decision tree classifier is trained. To investigate the influence of model complexity and overfitting additional experiments were performed by varying the maximum depth of the tree and applying cost-complexity pruning.

This exercise offers a defined framework that provides insights into how decision trees learn from training data and the effect of different parameters on their learning.

5. Implementation

we used the DecisionTreeClassifier from scikit-learn library to implement the decision tree model. We fed our model the training part of Iris dataset, and it learned decision rules based on feature values.

Figure 2: Decision boundary showing how the model partitions the feature space.



A baseline decision tree was first created with default parameters. The model recursively partitioned the data by selecting features and thresholds which optimised for minimising impurity as defined by the selected splitting criterion. Gini index and entropy were both explored in separate experiments to assess their respective effects on model performance.

Next, to approximate how the decision tree divides the feature space, a visualisation of the decision boundary using two randomly chosen features from the dataset. In this way, the behaviour of the model can be visualised in a two-dimensional space, each region representing a predicted class.

The decision boundary shows the separation of areas in feature space created by axis-parallel splitting. In contrast to linear models that learn all rectangular decision regions, the decision trees are not limited to this and can learn the non-linear patterns in your data.

We further assessed the model performance in terms of classification accuracy on test set apart from visualisation. This gives you a quantitative measure of how well the model generalizes to unseen data.

In conclusion, this is how decision trees learn from data, how they form their decision rules and help us visualize the decisions they make — not just by scoring them numerically but also graphically.

6. Results and Visual Analysis

In this section we present the results from decision tree model, how it behaves for different visualisations. These findings give us greater insight into how model complexity, feature importance and evaluation metrics affect overall performance.

To investigate the impact of model complexity, the performance of decision tree was assessed with respect to different depth. At increasing depth, training accuracy increases smoothly (and eventually has near-perfect performance). Yet, the accuracy on test does not reflect this trend and it might saturates or reduces beyond certain depth. This means there is overfitting, the model just memorize training data but fails to generalize for unseen data.

Figure 3: Training and testing accuracy at different tree depths.

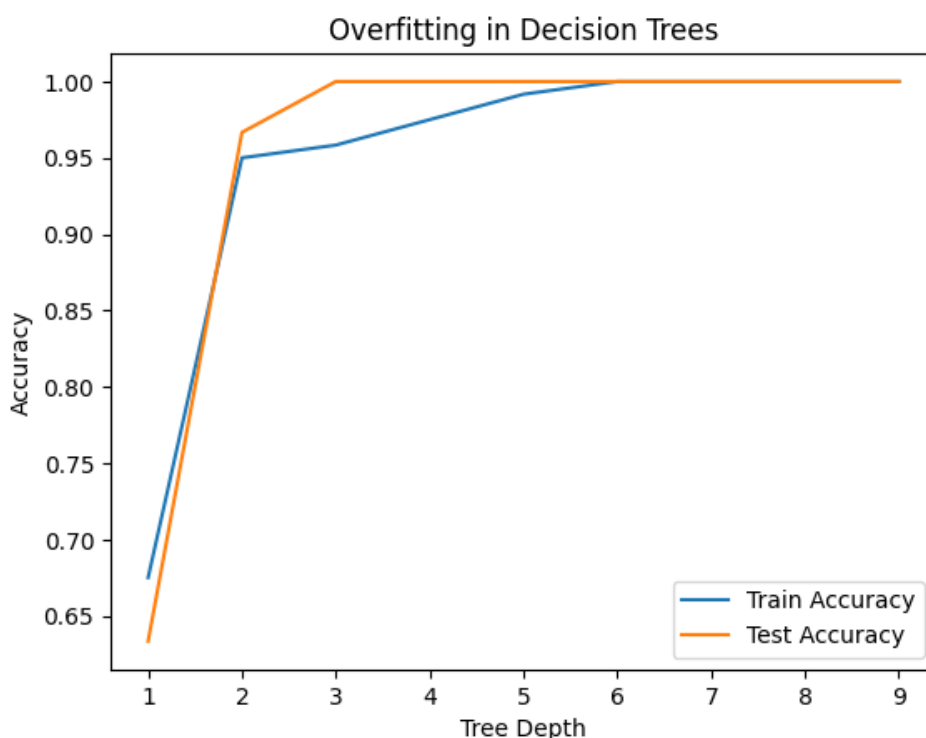
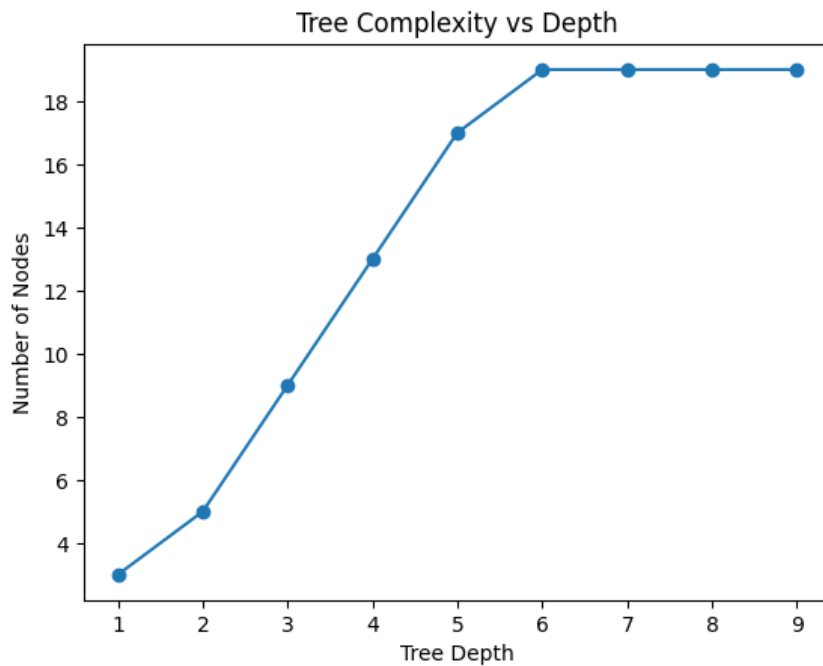
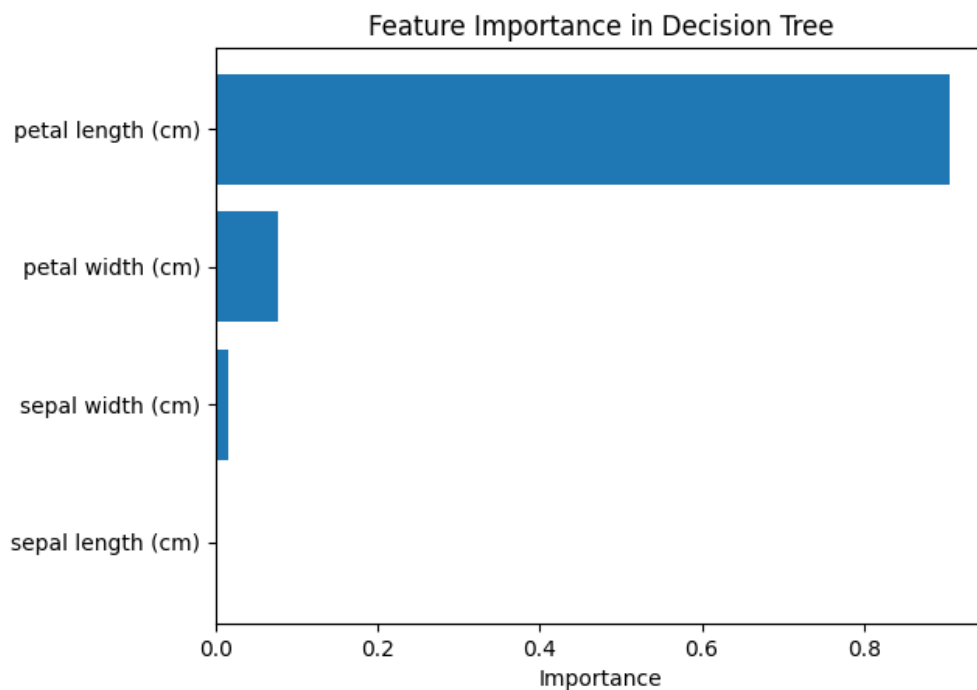


Figure 4: Growth in the number of nodes as tree depth increases.



Along with accuracy, the increase in model complexity was examined by comparing the number of nodes of different depths within the tree. This preserves the same 0-1 info model as the Tree of Life: for a given number of leaves, the number of nodes expands exponentially -- indicating how much more complicated trees get at greater depth. This additional complexity leads to overfitting and decreases interpretability.

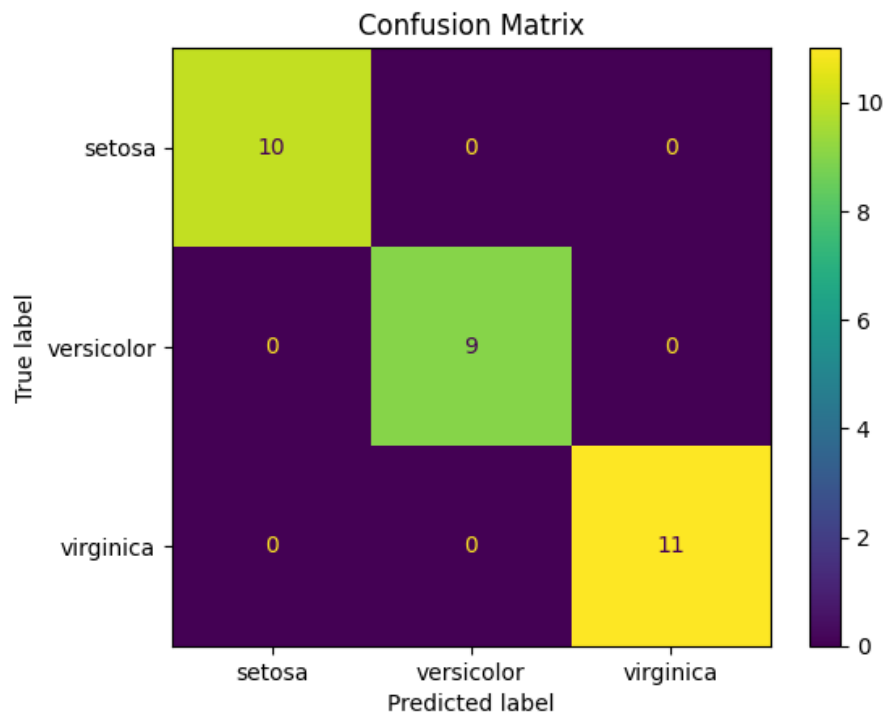
Figure 5: Feature importance scores indicating the contribution of each feature.



Feature Importance Analysis: The importance of each feature in the model's decision-making process was analyzed. The results show that some features like petal length and petal width have more influence on

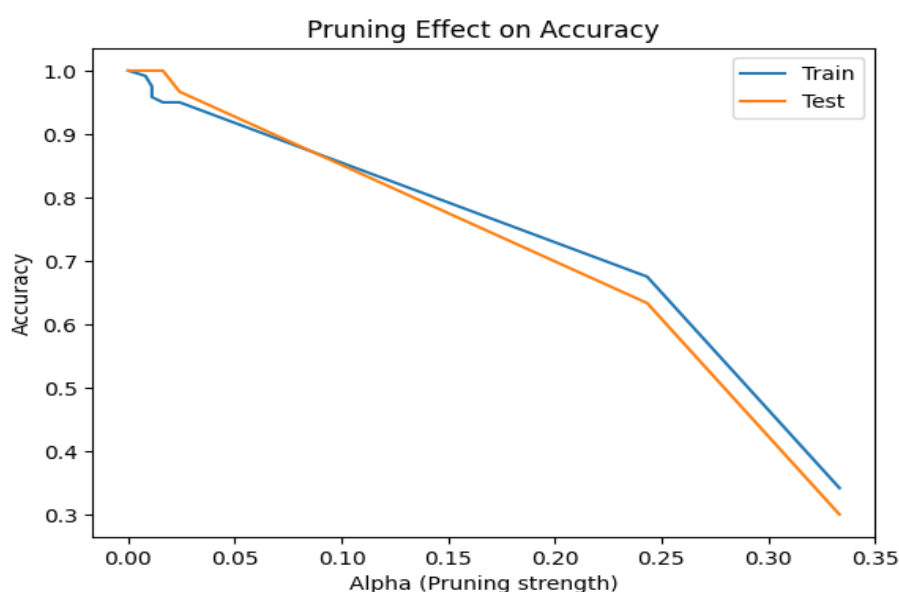
classification than others. Then, this shows how decision trees favour features that provide the most information in a given split.

Figure 6: Confusion matrix showing classification performance across classes.



To understand the model's performance in detail, we used a confusion matrix that gives us insight into correct and incorrect predictions for each class. The figures indicate the model is performing well overall and most samples are classified correctly, but some misclassifications can happen between similar classes.

Figure 7: Effect of pruning strength on model accuracy.



Last, the influence of cost-complexity pruning was examined. With the growing parameter of pruning, it let model to become simply and avoid overfitting. But this can bias commonly used classifiers by producing an

unsuitable model that is too simple, leading to underfitting. This shows you how to balance model complexity for potentially better performance.

In conclusion, the results demonstrate a crucial trade-off between model complexity and generalisation capabilities, akin to what has been documented in many other predictive models—as such they necessitate careful parameter tuning for decision tree-based models.

7. Overfitting and Pruning

Overfitting is one of the biggest issues of decision trees. Overfitting happens when a model memorizes not only the structure of the training data but also its noise and random fluctuations. Consequently, model fits the training data very well but has poor predictions on unseen samples.

In the case of decision trees, overfitting usually takes place because the tree becomes too deep. A big tree keeps splitting until it can completely classify the training data, and so a very complex model that has many nodes. But although this is a tempting end goal, such an approach does not generalise well — as we saw from the previous results (where more depth doesn't lead to better accuracy when tested on unseen data).

Exception design tree is not Pruned but pruning techniques helps in reducing the complexity. It removes branches that have little impact on enhancing the model's predictive freshness. A typical one is cost complexity pruning, which enters a parameter to punish the sculptures size.

By modifying the pruning parameter, a balance between model complexity and performance can be achieved. A low value leads to a complicated tree, and high value produces a simpler tree. As we also show in the pruning analysis, moderate pruning potentially improves generalisation, due to less overfitting, while excessive pruning can increase underfitting.

Thus, pruning is the key to fighting overfitting in decision tree models while making them accurate and generalisable.

8. Advantages and Limitations

Decision trees have few benefits, which make them an interesting option regarding machine learning. One of the key strengths they have over the limit models is interpretability. It is simple to visualise as a tree, making it straightforward for users to look at the path of decisions that led to a prediction. This allows decision trees to be especially useful for applications where interpretability is a key concern.

Decision trees also have the advantage of requiring very little data pre-processing. They can work with both numerical and categorical data, and do not require feature scaling. Furthermore, decision trees can capture non-linear relationships between feature variables and target variables.

But decision trees also have significant drawbacks. Another major disadvantage is that decision trees are prone to overfitting when the tree becomes too deep. This can lead to poor generalisation on unseen data. Decision trees are also sensitive to how the data is arranged, which may result in entirely different trees being created.

Additionally, when working with imbalanced datasets decision trees can be biased in favour of classes that have more samples to train on. This often requires using additional techniques, such as pruning or an ensemble method, to perform better.

With these disadvantages human designers still will find decision trees algorithms powerful and popular methods especially when interpretability is more relevant than for complex systems.

9. Conclusion

This tutorial covers the basic concepts of decision trees in terms of theory and code implementation. It explained how decision trees build a model recursively by splitting data on feature values, using measures like Gini index and entropy.

Through analysis on the Iris dataset, it was concluded that decision trees have good interpretability and accuracy. We signaled and trends for accuracy, decision boundaries, feature importance etc helped us understand how the model operates in different scenarios.

The tutorial also addressed model complexity, particularly with respect to the overfitting often encountered in deep trees and pruning techniques that can avoid it and lead to improved generalisation. These findings highlight the trade-off between model accuracy and simplicity.

Decision trees give an efficient and easy intuitive method towards classification problem. Despite some limitations, their interpretability and versatility make them one of the key player in practical applications.

10. References

- Scikit-learn Documentation: Decision Trees
<https://scikit-learn.org/stable/modules/tree.html>
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
- Tan, P.-N., Steinbach, M., & Kumar, V. (2018). *Introduction to Data Mining*. Pearson.
- Towards Data Science – Decision Tree Tutorials
<https://towardsdatascience.com/>